

scripts/pipeline/phase-05-distribute.sh — User Guide

Last verified: 2026-08-26 (feature 002-build-test-distribute-pipeline, task T043 — gate-only slice; clause 8 re-lettered to eight conditions (A)-(H) by the LVA-120/LVA-147 amendment)

Inheritance: HelixConstitution §11.4.18 (script documentation mandate); Lava §6.AA clause 8 (Pipeline Distribution Path), §6.Z (Anti-Bluff Distribute Guard), §6.AK (Cycle-Coverage Device Gate), §6.AH/§6.AG (container-or-VM emulators), §6.J (Anti-Bluff)

Overview

This script is the **§6.AA clause 8 refusal gate**. It decides whether a pipeline run has earned the clause 8 "Pipeline Distribution Path" permission — the permission to go debug → release with no operator pause — and then, deliberately, **distributes nothing**.

Clause 8 grants that permission if and only if **eight conjunctive conditions (A)-(H)** hold. The clause closes with its own standing note:

Any future reading of this clause that treats a green run report as sufficient WITHOUT condition (C) having genuinely executed — against the release-variant artifact specifically, never inferred from the debug variant — is the §6.J bluff class this project has recorded three times (§6.Z, §6.AB, §6.AK), and is a violation of this clause, not a use of it.

Re-lettered 2026-08-26. The former condition (D) — cycle-coverage on BOTH channels — was **withdrawn, not renumbered**, and former (E)-(I) became (D)-(H). **There is no condition (I)**. Anything written before 2026-08-26 that cites clause 8 by letter must be read against the map (E)→(D), (F)→(E), (G)→(F), (H)→(G), (I)→(H). (A), (B), (C) are unchanged.

A permission that broad is only safe if the thing enforcing it exists, is tested, and refuses by default **first**. So the guard ships on its own, with its own hermetic suite, and the distribute call is deferred.

What it does NOT do (honest scope)

- It **never invokes** scripts/firebase-distribute.sh. Condition (E) is verified by *static inspection*. Executing it would attempt a real upload to Firebase App Distribution.
- It writes **no Distribution Record** and mutates report.json in **no** way. It is a read-only reader of the run, plus one verdict artifact of its own at <run>/phase-05/gate-verdict.json.
- It is **not wired** into scripts/pipeline-build-test-distribute.sh.

Usage

```
scripts/pipeline/phase-05-distribute.sh <run_id> [repo-path]  
[options]
```

Option	Meaning
--report <path>	override the report.json location
--firebase-distribute-script <path>	override the script condition (E) inspects
--schema <path>	override the run-report schema
--suspension-dir <path>	override the clause-8 suspension directory

--cycle-coverage-script and --changelog-root were **removed** on 2026-08-26 together with the condition that used them. An unknown flag is fatal, so passing either now exits 2 rather than being silently ignored.

Every option selects **where to look**. None can change a verdict.

There is no escape hatch

There is deliberately no --force, --bypass, --skip, --override or equivalent, **and none may be added**. §6.Z clause 6 is absolute: no bypass flag exists and none may be added. A gate with an escape hatch is not a gate. An unrecognized argument is a hard error, never silently ignored — silently ignoring a --force someone believed worked is its own failure mode.

A condition the script **cannot evaluate refuses**. It never passes on the grounds that it could not tell.

Exit codes

Code	Meaning
0	

Code	Meaning
	A distribution completed. Reserved and unreachable today — no distribute step is implemented, so nothing returns 0.
2	GATE REFUSED — one or more of FR-009 / (A)-(H) failed; or a usage/configuration error. Refusal is the default.
3	GATE QUALIFIED , and the distribute step is not implemented.

What is checked

FR-009 baseline (evaluated before (A)-(H))

Refuses unless every US1/US2 phase (build, test, install_boot, live_verify) is PASS, evidence_summary.rejected_by_anti_bluff is 0, and no individual Evidence Record carries result: FAIL or a non-validated anti_bluff_status. FR-009 requires reporting *exactly which* evidence blocked, so every offending record is **named individually** rather than counted. A run with zero Evidence Records refuses — there is no evidence to authorize a distribution with.

Record shape is checked first. Every *.json under the run directory (excluding report.json, gate-verdict.json, and anything under raw/) must be a well-formed Evidence Record per contracts/evidence-record.schema.json: all seven required fields present and non-empty, result within PASS|FAIL|SKIPPED, category within its enum, and anti_bluff_status matching ^(validated|REJECTED: .+)\$\$. A file that does not parse, or is missing a field, is **unevaluable** — it can be read neither as passing nor as failing — and refuses. A missing anti_bluff_status specifically means the independent validator (FR-004) never saw that record, which is not the same as a record it validated.

A phase name may legitimately appear **more than once**: the lava-api-go and :api-app live-verification scripts each append their own live_verify entry. Every entry under a required phase name must be PASS; the count is not constrained.

The eight conditions

Condition	What this script actually checks
(A) Run Report identity	Report exists, parses, and validates structurally against contracts/pipeline-run-report.schema.json (required properties, additionalProperties, types, enums, patterns, minimums, item schemas, and

Condition	What this script actually checks
(B) Unqualified pass	the all of PASS rule). <code>commit_sha</code> must equal <code>git rev-parse HEAD</code> now — a mismatch is a refusal, not tolerable staleness. Each Distribution Record's <code>version_code</code> is cross-checked against the matching <code>build_artifacts</code> entry. outcome is PASS; every <code>phases[]</code> entry is PASS; no record is FAIL; no record has a non-validated <code>anti_bluff_status</code>; and no record is SKIPPED in a category named by (C) or (D). A run with zero Evidence Records refuses here too: "every record validated" over an empty set is an unexamined pass, not an unqualified one.
(C) Device evidence for BOTH variants	All four variants — <code>app-debug</code>, <code>app-release</code>, <code>api-app-debug</code>, <code>api-app-release</code> — need a record whose own category field is <code>real-device-challenge</code> (the field, not merely the directory it sits in), that is PASS, validated, identifies a container-or-VM runner, and does not indicate <code>host-direct</code> or a live physical device (<code>\$6.AH/\$6.AG</code> — the negative test is applied first, so a record carrying both markers cannot launder itself). One record, one artifact: a record whose text names more than one of the four variants is credited to none of them — it does not say which artifact it ran against, and a single emulator run is not evidence for two different APKs. Clause 8(C): <code>release-variant</code> evidence "is NOT inferable from <code>debug-variant</code> evidence". The report must also carry a matching <code>build_artifacts</code> entry built from the run's commit.
(D) Live verification of every live surface	Every <code>live_verify</code> phase entry is PASS, and both live surfaces — the <code>lava-api-go</code> service and the on-device <code>api-app</code> — have their own PASSing record. Provenance: only records written under a <code>live_verify</code> phase's own <code>evidence_dir</code> count. A build-time record — a <code>go</code> build of <code>lava-api-go</code>, a JVM unit test naming <code>api-app</code> — starts no service and satisfies nothing here, whatever it names. Device-challenge records are excluded for the same reason. A record naming both surfaces is credited to neither: one probe is not two surfaces exercised, whatever its summary says.
(E) Unmodified distribute path	<code>scripts/firebase-distribute.sh</code> exists, is tracked and <code>git-clean</code>, still carries its own Phase-1 gate markers, and defines no <code>bypass-shaped</code> option. Inspected, never executed.
(F) Scope	Branch is <code>master</code>, working tree is clean, and the report's precondition phase is PASS (FR-000).

Condition	What this script actually checks
(G) Disclosed residual gap	Every one of the four Android variants has a build_artifacts[] entry carrying a non-empty build_output_path — the property that names the locally-built file this run's evidence exercised, which is why the Firebase-delivered install is unverified. A report carrying a top-level residual-gap property is refused: that field was retired on 2026-08-26 and the schema forbids it. This is a disclosure check, not a verification of the release build ; the substantive release-variant protection is (C).
(all) Recorded verdict	The verdict artifact at <run>/phase-05/gate-verdict.json must actually be written. An authorization the gate cannot record leaves clause 8 with no audit trail, so a failed write adds a refusal; it can never remove one.
(H) Automatic suspension trigger	No ACTIVE marker of any kind under .lava-ci-evidence/clause-8-suspension/, and no incident under .lava-ci-evidence/sixth-law-incidents/ recording a clause-8 suspension without an affirmative operator re-authorization. The re-authorization value is read, not merely the key: "reauthorized": false is a denial and keeps the suspension, and a free-text note never grants. If the suspension directory, the marker, or an incident file exists but cannot be read or parsed, the condition is unevaluable and refuses — it never reads as "not suspended". Suspension is cleared only by an explicit recorded act, never by age.

Honest defects this gate surfaces rather than papers over

1. Nothing verifies per-variant cycle coverage. Not this gate either.

Until 2026-08-26 a condition (D) required the \$6.AK cycle-coverage check to pass for the debug channel **and** the release channel. It was **withdrawn**, for two independently sufficient reasons.

Its premise was false. The condition existed because scripts/firebase-distribute.sh resolved the \$6.AK channel to debug for any MODE other than exactly release, via a *) AK_CHANNEL="debug" catch-

all. That catch-all is gone: MODE has exactly two reachable values and an explicit two-arm case hard-errors on a third (LVA-120 retired the combined mode).

And it never achieved what it claimed. In scripts/check-cycle-coverage.sh, CHAN is assigned by the --channel parse arms (lines 37–38) and asserted non-empty (line 51) — and never read again. Both artifacts it reads, the cycle-coverage-map and the §6.Z evidence file, resolve from \$EDIR + \$VER alone. There is no eval, no \${!indirect}, no declare -n. --channel=debug, --channel=release and --channel=banana produce **byte-identical output** against identical fixtures. Repairing the condition to "invoke --channel=release" would have mandated a call that provably does nothing.

Stated plainly, because withdrawing the condition does not create the check it was reaching for: no release-variant coverage artifacts exist — the evidence tree under .lava-ci-evidence/distribute-changelog/ partitions by **app**, not by build variant. Per-variant cycle coverage is **unverified**, is tracked as **§6.AA-pipeline-debt-D**, and the release-variant protection inside clause 8 rests **entirely** on condition (C). The gate prints this on every run rather than leaving it to be inferred from a condition that is no longer listed.

2. The (A)/residual-gap deadlock is resolved, and residual-gap now fails

Until 2026-08-26 condition (H) mandated a top-level residual-gap field on the run report, while contracts/pipeline-run-report.schema.json is "additionalProperties": false and defines no such property. Taken literally (A) and (H) could not both hold, so **no run could ever qualify** (LVA-147). This gate previously tolerated residual-gap as a single clause-8 extension to keep the clause operable.

That tolerance is removed. The condition — now **(G)** — was reworded to record the same gap through build_artifacts[].build_output_path, a property the schema already defines and already requires. No schema amendment was made and none is owed for this. A report carrying a top-level residual-gap property is now refused by (A) like any other unknown property, which is what the schema has always said.

The gap itself is **unchanged and still open**: the pipeline's evidence exercises locally-built artifacts and does not cover the Firebase upload → download → install path on a physical device. That remains **§6.AA-pipeline-debt**, OWED.

A third, smaller limitation worth stating

evidence-record.schema.json has **no artifact field**, so a record cannot structurally declare which variant it exercised. Condition (C) therefore binds a record to a variant by requiring the variant's artifact_id token to appear in the record's test_id, command, or assertion_summary. That binding is weaker than a typed field would be. It **errs toward refusal** — a record that does not say which artifact it ran against counts for no variant, and a record that names *several* also counts for none. Adding an artifact_id to the Evidence Record schema would close this.

A fourth: clause 8(H) is checked on two of its three limbs

Clause 8(H) lifts a suspension on three things: (i) a §6.Z-class incident record, (ii) a covering device Challenge that reproduces the failure RED-then-GREEN per §6.AK clause 2, and (iii) a written operator re-authorization. This gate mechanically checks **(i) and (iii)**. Limb (ii) has no machine-readable form in the incident convention, and inventing one here would assert a check that is not happening — it stays operator- and reviewer-verified. Stated rather than implied, so nobody reads a green (H) as proof the covering Challenge exists.

Output artifact

<run>/phase-05/gate-verdict.json records the run id, evaluation time, HEAD sha, the QUALIFIED/REFUSED verdict, the FR-009 result, every condition's verdict and detail, and a per_variant_cycle_coverage field stating that it is **NOT VERIFIED** (§6.AA-pipeline-debt-D). What the gate does not check lives in the artifact, not only on a terminal someone has since closed.

Tests

Two hermetic suites, both building disposable git fixtures and running nothing real.

tests/pipeline/test_phase_05_distribute_gate.sh asserts a fully-qualifying run **passes** (so a blanket refuse-everything implementation cannot pass the suite), that each condition mutated **alone** flips the verdict to REFUSED and names itself, that no plausible override argument forces a failing run through, that the gate never invokes the distribute script, that **no** condition (I) is

reported, that a variant missing its `build_output_path` is refused by (G), and that a report carrying the retired `residual-gap` property is refused by (A).

`tests/pipeline/test_phase_05_distribute_gate_vacuity.sh` attacks the other direction: for each condition, what happens when the thing it examines is absent, empty, unreadable, malformed, or merely *shaped like* the thing it should be. A gate that wrongly refuses is an annoyance; a gate that wrongly passes authorises an unattended release. Every case in that file was first observed returning `QUALIFIED` against an earlier revision of this script, so each is a regression test with a recorded `RED` — build-time records satisfying what is now (D), two debug records satisfying all four variants of (C), an incident recording `reauthorized: false` clearing what is now (H), `--schema` pointed at a non-object document turning (A) from `FAIL` into `PASS`, and an evidence file containing the literal text `this is not json` being counted among "N record(s) all validated". It carries its own positive controls so the fixes cannot degenerate into refusing everything.

`tests/pipeline/test_phase_05_distribute_fix_audit.sh` is the **second pair of eyes on the repairs themselves**. The nine fixes the two suites above describe were written and verified by the same agent that wrote the code they repaired, which makes 416 lines of unreviewed gate logic. Auditing them as unreviewed code found seven more defects — three of them reaching `QUALIFIED`:

#	Where	What was wrong	Direction
1	(C)	<code>_names_variant</code> answered "does this name <code>app-debug</code> ?" with no whenever <code>api-app-debug</code> appeared <i>at all</i> , including when the text named both — so the "a record naming more than one variant counts for none" guard never fired for that pair. Four records that all ran a client-app artifact satisfied all four variants, with zero <code>:api-app</code> device runs in the run.	wrongly PASSED
2	(A)	Refusing a <i>non-object</i> <code>--schema left {}</code> , an object that constrains nothing just as completely. A report carrying a property the real schema forbids went <code>REFUSED</code> → <code>QUALIFIED</code> , with (A) asserting "report schema-valid".	wrongly PASSED
3	verdict artifact	Condition rows were <code>printf</code> 'd as TAB-separated lines and split back apart, and details quote report content verbatim — so a report value carrying a newline and a tab injected rows : 11 rows for the 9 conditions clause 8 then had, and a duplicate whose last	forged record

#	Where	What was wrong	Direction
		value read PASS. The clause 8 audit record was forgeable from the artifact it audits.	
4	now (H)	Reading the re-authorization value fixes nothing for an incident the scan never reaches. "\$6.AA clause 8(I) suspension is ACTIVE" (fixture text quoted verbatim from before the 2026-08-26 re-lettering; the gate matches "clause 8" plus a suspension word in any order, so no letter is load-bearing) carrying "operator_reauthorization": "none" — an explicit written denial — was invisible, and the run qualified.	wrongly PASSED
5	now (D)	Over-corrected. The :api-app is a client of lava-api-go, so its live-verification summary names the service it reached; crediting a both-naming record to <i>neither</i> surface refused an honest run and reported the api-app surface had no evidence when it did.	wrongly REFUSED
6	now (F)	Carried the exact defect FR-009 was repaired for — select(.name=="precondition") .result unaggregated concatenates two entries into PASS\nPASS. The repair was applied at one site and not swept.	wrongly REFUSED
7	--help	Still shape-coupled. The awk range ended at the first horizontal rule, so a separator rule added <i>inside</i> the Usage block silently dropped the exit codes, and shortening the rules printed 1316 lines.	cosmetic

Each has a recorded RED and a positive control. The controls are the point: case 2P proves --schema still works against the real contract schema, 4P proves broadening what is now (H) does not start refusing on the existing incident corpus (the whole real .lava-ci-evidence/sixth-law-incidents/ tree is copied into a fixture and must not refuse), and 5P proves one record is still never two live surfaces. Without them, "refuse unconditionally" would pass the suite — which is how an anti-bluff gate suite becomes a bluff.

The TODO that remains

The distribute step itself. When it lands it must invoke `firebase-distribute.sh` unmodified **twice** — once `--debug-only`, then once `--release-only` — with all its Phase-1 gates active (never `firebase appdistribution:distribute` directly, which is a §6.Z violation whether or not a pipeline is running, and never a combined invocation, which no longer exists), write one Distribution Record per uploaded artifact whose `version_code` equals the `versionCode` compiled into that artifact, append its result via `append_phase_result`, and define what happens when the debug stage succeeds and the release stage refuses — a partially-distributed cycle is now reachable in a way the retired combined invocation never made it, and that semantics is **undecided**.

Until then the script exits non-zero even when the gate qualifies: a gate that returned success while distributing nothing would be its own small bluff.